

# ADR-001: Expansão ODIN-ETL para Nordeste

Equipe ODIN

Julho 2026

## Sumário

|                                                                                   |          |
|-----------------------------------------------------------------------------------|----------|
| <b>ADR-001: Expansão do ODIN-ETL para Todos os Estados do Nordeste</b>            | <b>2</b> |
| 1. Contexto                                                                       | 2        |
| 1.1 Escala da Expansão                                                            | 2        |
| 2. Decisão                                                                        | 2        |
| 2.1 Princípios de Design                                                          | 2        |
| 3. Análise de Impacto por Componente                                              | 3        |
| 3.1 Componentes sem alteração necessária                                          | 3        |
| 3.2 Componentes com alteração trivial (config only)                               | 3        |
| 3.3 Componentes com alteração moderada                                            | 3        |
| 3.4 Componentes com alteração complexa (refactor)                                 | 3        |
| 4. Riscos e Mitigações                                                            | 3        |
| 5. Alternativas Consideradas                                                      | 4        |
| 5.1 Processar Brasil inteiro (rejeitada)                                          | 4        |
| 5.2 Banco por estado separado (rejeitada)                                         | 4        |
| 5.3 Expansão incremental estado por estado (aceita parcialmente)                  | 4        |
| 6. Plano de Execução por Sprint                                                   | 4        |
| Estimativa Total: 4 sprints (2 semanas cada = 8 semanas)                          | 4        |
| 7. Sprint 1 — Configuração e Multi-UF no Socioeconômico                           | 4        |
| Task 1.1: Parametrizar <code>filtro_uf</code> no IBGE Censo para lista de UFs     | 5        |
| Task 1.2: Atualizar filenames nos transforms do socioeconômico                    | 5        |
| Task 1.3: Atualizar loads do socioeconômico para multi-UF                         | 6        |
| Task 1.4: Atualizar o <code>config/ibge_censo.yml</code> com validações genéricas | 6        |
| Task 1.5: Atualizar script de validação                                           | 6        |
| 8. Sprint 2 — Geodados Multi-Estado                                               | 6        |
| Task 2.1: Refatorar <code>geo_ingest_pipeline</code> para multi-UF                | 7        |
| Task 2.2: Atualizar <code>bairro_pipeline</code> para usar GeoPackage consolidado | 7        |
| Task 2.3: Atualizar <code>setor_pipeline</code> para GeoPackage consolidado       | 8        |
| Task 2.4: Atualizar <code>municipio_pipeline</code> com GeoJSON multi-estado      | 8        |
| 9. Sprint 3 — Indicadores Educacionais Multi-UF + CEP                             | 8        |
| Task 3.1: Expandir filtro do <code>geocode_pipeline</code> para NE                | 8        |
| Task 3.2: Expandir INEP indicadores para NE                                       | 9        |
| Task 3.3: Adquirir e integrar dataset de CEPs do Nordeste                         | 9        |
| Task 3.4: Rodar pipeline educacional end-to-end para NE                           | 10       |
| 10. Sprint 4 — Validação, Testes, Otimização e Deploy                             | 10       |
| Task 4.1: Criar testes para os novos indicadores e multi-UF                       | 10       |
| Task 4.2: Otimizar spatial joins para performance                                 | 11       |
| Task 4.3: Atualizar Makefile e Docker para NE                                     | 11       |
| Task 4.4: Script de validação multi-estado                                        | 11       |
| Task 4.5: Deploy em produção                                                      | 12       |
| 11. Diagrama de Dependências entre Tasks                                          | 12       |
| 12. Custos Estimados                                                              | 12       |

|                                      |    |
|--------------------------------------|----|
| 13. Critérios de Aceitação . . . . . | 12 |
| 14. Referências . . . . .            | 13 |

# ADR-001: Expansão do ODIN-ETL para Todos os Estados do Nordeste

**Status:** Proposto

**Data:** 2026-07-10

**Autores:** Equipe ODIN

## 1. Contexto

O ODIN-ETL atualmente processa dados educacionais e socioeconômicos exclusivamente para a **Paraíba (PB)**. A aplicação consome dados do Censo Escolar INEP, indicadores educacionais (IDEB, AFD, TDI, etc.), dados socioeconômicos do Censo Demográfico IBGE 2022, e malhas geográficas (bairros, setores censitários, municípios).

A decisão estratégica é expandir a cobertura para todos os **9 estados da região Nordeste**: Maranhão (MA), Piauí (PI), Ceará (CE), Rio Grande do Norte (RN), Paraíba (PB), Pernambuco (PE), Alagoas (AL), Sergipe (SE) e Bahia (BA).

### 1.1 Escala da Expansão

| Métrica                    | PB (atual) | Nordeste (9 UFs)      | Fator de crescimento |
|----------------------------|------------|-----------------------|----------------------|
| Escolas (públicas)         | 3.737      | ~75.000               | 20x                  |
| Municípios                 | 223        | 1.794                 | 8x                   |
| Setores censitários        | 9.639      | ~180.000              | 19x                  |
| Bairros (com dados IBGE)   | 194        | ~3.000                | 15x                  |
| Documentos MongoDB (total) | ~13.000    | ~260.000              | 20x                  |
| Dados no Bronze (IBGE)     | ~30 MB     | ~30 MB (mesmo ZIP BR) | 1x                   |
| Dados no Silver            | ~50 MB     | ~1 GB                 | 20x                  |
| Gold parquets              | ~5 MB      | ~100 MB               | 20x                  |

## 2. Decisão

Expandir o ETL para processar os 9 estados do Nordeste **sem alterar a arquitetura fundamental** (Medallion: Bronze → Silver → Gold → MongoDB). A expansão será feita por **parametrização** dos filtros de UF e **iteração por estado** nos pipelines que dependem de dados geográficos estaduais (shapefiles).

### 2.1 Princípios de Design

1. **Configuração sobre código:** filtros de UF vivem em YAMLS, não hardcoded
2. **Processamento por UF:** shapefiles e GeoPackages por estado; dados tabulares filtrados por lista de UFs
3. **Schema MongoDB inalterado:** os documentos já são state-agnostic (campo `sg_uf` presente)
4. **Idempotência mantida:** re-execuções não criam duplicatas
5. **Retrocompatibilidade:** dados da PB continuam funcionando durante a migração

### 3. Análise de Impacto por Componente

#### 3.1 Componentes sem alteração necessária

| Componente                          | Motivo                                                                    |
|-------------------------------------|---------------------------------------------------------------------------|
| censo_pipeline                      | Já filtra por <code>NO_REGIAO == "NORDESTE"</code> — processa 75k escolas |
| MongoDB schema/indexes              | Chaves únicas são IDs IBGE (state-agnostic)                               |
| document_builder.py                 | Constrói documento por escola — sem filtro de UF                          |
| geo_utils.py / calcular_indicadores | Lógica genérica por grupo — sem UF                                        |
| storage.py                          | Backend de I/O — sem lógica de negócio                                    |

#### 3.2 Componentes com alteração trivial (config only)

- **Filtro geocode** — `config/config_geocode.yml` — mudar `filtro_uf` para lista de 9 UFs
- **Filtro INEP indicadores** — `config/inep_indicadores.yml` — mudar `filtro_uf` para lista
- **Filtro IBGE censo** — `config/ibge_censo.yml` — mudar `filtro_uf` para lista de códigos
- **Docker resources** — `docker-compose.prod.yml` — ETL memory 4G para 8G

#### 3.3 Componentes com alteração moderada

- **Geocode extract** — `geocode_pipeline/etl/extract.py` — renomear output para `escolas_nordeste.parquet`
- **INEP indicadores extract** — `inep_indicadores_pipeline/etl/extract.py` — filtro `.isin(lista)` em vez de `== "PB"`
- **INEP transform validação** — `inep_indicadores_pipeline/etl/transform.py` — threshold de 3k para 30k
- **CEP lookup** — `data/gold/cep.json` — adquirir dataset de CEPs do NE inteiro
- **Validação** — `scripts/validar_socioeconomico.py` — parametrizar ou remover thresholds PB
- **Município GeoJSON** — `municipio_pipeline/etl/transform.py` — carregar GeoJSON multi-estado

#### 3.4 Componentes com alteração complexa (refactor)

- **Geo ingest pipeline** — `geo_ingest_pipeline/etl/extract.py` — loop por 9 UFs, download shapefiles, merge GeoPackage
- **IBGE censo extract** — `ibge_censo_pipeline/etl/extract.py` — multi-UF filter, file naming dinâmico
- **IBGE censo transforms (3)** — `municipio/setor/bairro transform.py` — inputs dinâmicos, validações generalizadas
- **IBGE censo loads (3)** — `municipio/setor/bairro load.py` — GPKG paths dinâmicos por UF
- **Bairro/Setor aggregation** — `bairro_pipeline` e `setor_pipeline transform` — multi-state GeoPackage, remover filter PB

---

### 4. Riscos e Mitigações

| Risco                                                         | Probabilidade | Impacto | Mitigação                                                                 |
|---------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------|
| Geocodificação de ~25k escolas novas sem coordenada           | Alta          | Alto    | Usar coordenadas do INEP (Base dos Dados) como fallback antes de API paga |
| RAM insuficiente para spatial join 180k setores × 75k escolas | Média         | Médio   | Processar por estado (chunk por UF) no spatial join                       |
| FTP do IBGE indisponível durante download                     | Média         | Baixo   | Retry com backoff já implementado + cache Bronze                          |

| Risco                                                | Probabilidade | Impacto | Mitigação                                                     |
|------------------------------------------------------|---------------|---------|---------------------------------------------------------------|
| Dataset de CEPs incompleto para estados menores      | Média         | Médio   | Fallback por município via <code>CO_MUNICIPIO</code> do censo |
| Tempo de execução do pipeline > 1h                   | Alta          | Baixo   | Paralelizar downloads; spatial join por UF                    |
| Shapefiles do IBGE com encoding diferente por estado | Baixa         | Médio   | Já usamos <code>encoding="latin-1"</code> que cobre todos     |

## 5. Alternativas Consideradas

### 5.1 Processar Brasil inteiro (rejeitada)

- Volume 5x maior que NE sem benefício imediato
- Custo de geocodificação proibitivo (~\$1500 one-time)
- Shapefiles de setores de SP/MG são enormes (memória)

### 5.2 Banco por estado separado (rejeitada)

- Complexidade operacional desnecessária
- MongoDB suporta 260k docs sem degradação
- Índice por `sg_uf` suficiente para queries filtradas

### 5.3 Expansão incremental estado por estado (aceita parcialmente)

- Sprint 1 valida com 1 estado adicional (PE ou CE) antes de rodar tudo
- Reduz risco de bugs ocultos em estados com dados atípicos

## 6. Plano de Execução por Sprint

**Estimativa Total: 4 sprints (2 semanas cada = 8 semanas)**

| Sprint | Foco                                     | Entregável                                               |
|--------|------------------------------------------|----------------------------------------------------------|
| 1      | Configuração e infraestrutura            | Pipeline roda para NE completo (sem shapefiles)          |
| 2      | Geodados multi-estado                    | Shapefiles de 9 UFs ingeridos, spatial joins funcionando |
| 3      | Indicadores INEP multi-UF + CEP          | Dados educacionais completos para NE                     |
| 4      | Validação, testes, documentação e deploy | Pipeline em produção para 9 estados                      |

## 7. Sprint 1 — Configuração e Multi-UF no Socioeconômico

**Objetivo:** Pipeline IBGE Censo rodando para os 9 estados do NE.

**Duração:** 2 semanas

**Pré-requisitos:** Nenhum (pode começar imediato)

### Task 1.1: Parametrizar `filtro_uf` no IBGE Censo para lista de UFs

**Arquivo a alterar:** `config/ibge_censo.yml`

**O que fazer:** - Mudar `filtro_uf: "25"` para `filtro_uf: ["21","22","23","24","25","26","27","28","29"]`

**Arquivo a alterar:** `src/jobs/socioeconomico_jobs/ibge_censo_pipeline/etl/extract.py`

**O que fazer:** - A variável `FILTRO_UF` é uma string. Mudar para lista. - Na função `_filtrar_uf()`, onde faz `df[coluna] == FILTRO_UF`, mudar para `df[coluna].isin(FILTRO_UF)` (para `CD_UF`) e `df[coluna].str[:2].isin(FILTRO_UF)` (para `CD_MUN/CD_SETOR`). - Na função `_salvar_silver()`, mudar o sufixo de `_pb.parquet` para `_nordeste.parquet`.

**Por que:** O extract baixa o ZIP do Brasil inteiro mas filtra só a PB no Silver. Expandir o filtro dá NE completo sem downloads adicionais.

**Como testar:**

```
uv run python -c "  
from src.jobs.socioeconomico_jobs.ibge_censo_pipeline.etl.extract import baixar_dataset  
path = baixar_dataset('basico', 'municipio')  
import pandas as pd  
df = pd.read_parquet(path)  
print(f'Municípios: {len(df)}') # Esperado: 1794  
print(f'UFs:', df['CD_UF'].unique() if 'CD_UF' in df.columns else 'sem CD_UF')  
"
```

**O que estudar:** - Pandas `.isin()` para filtros com múltiplos valores - Estrutura do CSV do IBGE (colunas `CD_UF`, `CD_MUN` com prefixo de UF)

---

### Task 1.2: Atualizar filenames nos transforms do socioeconômico

**Arquivos a alterar:**

- `src/jobs/socioeconomico_jobs/ibge_censo_pipeline/etl/municipio/transform.py`
- `src/jobs/socioeconomico_jobs/ibge_censo_pipeline/etl/setor/transform.py`
- `src/jobs/socioeconomico_jobs/ibge_censo_pipeline/etl/bairro/transform.py`

**O que fazer em cada:**

1. Mudar `_SILVER_INPUTS` — trocar sufixo `_pb.parquet` por `_nordeste.parquet`
2. Mudar `GOLD_OUTPUT` — trocar `_pb.parquet` por `_nordeste.parquet`
3. Remover/generalizar `df["uf"] = "PB"` — extrair UF real da coluna `CD_MUN` (primeiros 2 dígitos mapeados para sigla)
4. Remover validações hardcoded (`MUNICIPIOS_ESPERADOS = 223`, `SETORES_MIN/MAX`)

**Arquivo auxiliar a criar:** `src/common/ibge_codes.py`

```
CODIGO_UF_PARA_SIGLA = {  
    "21": "MA", "22": "PI", "23": "CE", "24": "RN", "25": "PB",  
    "26": "PE", "27": "AL", "28": "SE", "29": "BA",  
}
```

**Por que:** Os filenames atuais são PB-specific e o transform quebraria ao receber dados de outros estados.

**Como testar:**

```
uv run python -m src.jobs.socioeconomico_jobs.ibge_censo_pipeline.etl.municipio.transform  
# Deve processar 1794 municípios sem erros de validação
```

**O que estudar:** - Códigos IBGE de UF: <https://www.ibge.gov.br/explica/codigos-dos-municipios.php> - Data-class `TransformResult` e como os avisos são usados

### Task 1.3: Atualizar loads do socioeconômico para multi-UF

**Arquivos a alterar:** - `src/jobs/socioeconomic_jobs/ibge_censo_pipeline/etl/municipio/load.py` - `src/jobs/socioeconomic_jobs/ibge_censo_pipeline/etl/setor/load.py` - `src/jobs/socioeconomic_jobs/ibge_censo_pipeline`

**O que fazer:** - Nos loads de setor e bairro: o `_GPKG_PATH` aponta para `setores_pb.gpkg` / `bairros_pb.gpkg`. Mudar para ler configuração dinâmica ou path genérico (`setores_nordeste.gpkg`). - Os loads já são state-agnostic no upsert (chave é `cd_setor`, `cd_bairro`, `municipioIdIbge`). Só precisam do GPKG path correto.

**Como testar:**

```
uv run python -m src.jobs.socioeconomic_jobs.main
# Deve rodar extract → transform → load para 1794 municípios
```

**O que estudar:** - Como o `pymongo.UpdateOne` com `$setOnInsert` funciona - Padrão de upsert idempotente

---

### Task 1.4: Atualizar o `config/ibge_censo.yml` com validações genéricas

**Arquivo a alterar:** `config/ibge_censo.yml`

**O que fazer:** - Remover `transform_municipio.municipios_esperados: 223` - Ou mudar para 1794 (NE completo) - Ajustar `pop_total_min/max` para `50_000_000` / `60_000_000` (pop NE ~57M) - Ajustar `contratos_qualidade: total_populacao.max: 900.000` → `3.000.000` (Salvador tem ~2.9M) - Os demais percentuais (0-100%) continuam válidos - Renomear `silver_inputs` para usar `_nordeste.parquet`

**Por que:** Validações calibradas para PB quebram com dados de outros estados (Salvador é 3x maior que JP).

**Como testar:** Os logs do transform mostram avisos de qualidade — devem voltar zerados.

**O que estudar:** - Dados populacionais do NE no IBGE (para calibrar thresholds) - Diferença entre erro (falha) e aviso (log warning) no pipeline

---

### Task 1.5: Atualizar script de validação

**Arquivo a alterar:** `scripts/validar_socioeconomico.py`

**O que fazer:** - Mudar paths Gold para `_nordeste.parquet` - Remover `MUNICIPIOS_ESPERADOS = 223` → usar 1794 - Remover `POP_TOTAL_MIN/MAX` de PB → usar range do NE - Remover benchmark João Pessoa (`JP_ID`) ou adicionar benchmarks por estado (Recife, Salvador, Fortaleza) - Mudar pattern Silver para `_nordeste.parquet`

**Como testar:**

```
uv run python scripts/validar_socioeconomico.py
```

**O que estudar:** Nada novo — é ajuste de constantes.

---

## 8. Sprint 2 — Geodados Multi-Estado

**Objetivo:** Shapefiles de bairros, setores e municípios dos 9 estados ingeridos e prontos para spatial join.

**Duração:** 2 semanas

**Pré-requisitos:** Sprint 1 concluída

---

## Task 2.1: Refatorar `geo_ingest_pipeline` para multi-UF

**Arquivo a alterar:** `src/jobs/education_jobs/geo_ingest_pipeline/etl/extract.py`

**O que fazer:** 1. Remover constantes hardcoded (`BAIRROS_SHP_DIR`, `BAIRROS_SHP_FILE`, paths de PB) 2. Criar mapeamento de UFs com URLs de download no config:

```
# config/config_geocode.yml → geo_pipeline.ibge.estados
estados:
  MA: { codigo: "21", sigla_lower: "ma" }
  PI: { codigo: "22", sigla_lower: "pi" }
  CE: { codigo: "23", sigla_lower: "ce" }
  RN: { codigo: "24", sigla_lower: "rn" }
  PB: { codigo: "25", sigla_lower: "pb" }
  PE: { codigo: "26", sigla_lower: "pe" }
  AL: { codigo: "27", sigla_lower: "al" }
  SE: { codigo: "28", sigla_lower: "se" }
  BA: { codigo: "29", sigla_lower: "ba" }
```

3. Implementar loop que para cada UF:

- Baixa shapefile de setores do GeoFTP IBGE
- Baixa shapefile de bairros do GeoFTP IBGE
- Baixa shapefile de municípios do GeoFTP IBGE
- Salva cada GeoPackage no Silver: `{uf_lower}_setores.gpkg`, `{uf_lower}_bairros.gpkg`

4. Ao final, criar GeoPackage consolidado: `setores_nordeste.gpkg`, `bairros_nordeste.gpkg`

**Arquivo novo a criar:** `src/jobs/education_jobs/geo_ingest_pipeline/config/estados_nordeste.py`

```
ESTADOS_NORDESTE = {
    "MA": {"codigo_ibge": "21", "nome": "Maranhão"},
    "PI": {"codigo_ibge": "22", "nome": "Piauí"},
    # ... todos os 9
}
IBGE_GEOFTP_BASE = "https://geoftp.ibge.gov.br/organizacao_do_territorio/malhas_territoriais"
```

**Por que:** O pipeline atual processa apenas shapefiles da PB que foram colocados manualmente no bronze. Precisamos automatizar o download e processamento para 9 UFs.

**Como testar:**

```
uv run python -m src.jobs.education_jobs.geo_ingest_pipeline.main
# Deve gerar 9 GeoPackages de setores + 9 de bairros + consolidado
ls data/silver/*_setores.gpkg data/silver/*_bairros.gpkg
```

**O que estudar:** - GeoPandas `read_file()` com ZIPs: `gpd.read_file("zip://arquivo.zip")` - `pd.concat()` para GeoDataFrames: manter CRS consistente - Estrutura de URLs do IBGE GeoFTP (explorar manualmente: `ftp://geoftp.ibge.gov.br/`) - Encodings de shapefiles brasileiros (latin-1 / utf-8 / cp1252)

---

## Task 2.2: Atualizar `bairro_pipeline` para usar GeoPackage consolidado

**Arquivo a alterar:** `src/jobs/education_jobs/bairro_pipeline/etl/transform.py`

**O que fazer:** 1. Mudar `BAIRROS_GPKG = "data/silver/bairros_pb.gpkg" → "data/silver/bairros_nordeste.gpkg"`  
2. **Remover** o filtro `df_censo[df_censo["SG_UF"] == "PB"]` (linha ~108) — agora queremos todas as UFs 3. O `spatial join` já é genérico — escola dentro de bairro independe do estado

**Como testar:**

```
uv run python -m src.jobs.education_jobs.bairro_pipeline.main
# Deve associar ~3000 bairros com escolas (vs 194 antes)
```

**O que estudar:** - Como spatial join (`gpd.sjoin`) funciona: ponto dentro de polígono - Por que escolas fora de polígonos são esperadas (áreas rurais sem bairro oficial)

---

### Task 2.3: Atualizar `setor_pipeline` para GeoPackage consolidado

**Arquivo a alterar:** `src/jobs/education_jobs/setor_pipeline/etl/transform.py`

**O que fazer:** 1. Mudar `SETORES_GPKG = "data/silver/setores_pb.gpkg" → "data/silver/setores_nordeste.gpkg"`  
2. O spatial join já funciona sem filtro de UF

**Atenção de performance:** 180k setores × 75k escolas pode exigir ~4GB RAM. Se necessário, processar por UF:

```
for uf in UFS_NORDESTE:
    gdf_setores_uf = gdf_setores[gdf_setores["CD_UF"] == uf]
    gdf_escolas_uf = gdf_escolas[gdf_escolas["SG_UF"] == uf]
    resultado_uf = gpd.sjoin(gdf_escolas_uf, gdf_setores_uf, ...)
```

**Como testar:**

```
uv run python -m src.jobs.education_jobs.setor_pipeline.main
# Deve associar ~50k setores com escolas
```

**O que estudar:** - Memory profiling com `tracemalloc` ou `memory_profiler` - Estratégias de chunking para spatial joins grandes

---

### Task 2.4: Atualizar `municipio_pipeline` com GeoJSON multi-estado

**Arquivo a alterar:** `src/jobs/education_jobs/municipio_pipeline/etl/transform.py`

**O que fazer:** 1. Mudar `MUNICIPIOS_GEOJSON = "data/silver/geojs-25-mun (1).json"` → carregar GeoJSON de todos os estados 2. Opção A: baixar GeoJSON consolidado do IBGE API (<https://servicodados.ibge.gov.br/api/v3/malhas/es> para NE) 3. Opção B: usar os shapefiles de municípios já baixados no Task 2.1

**Arquivo novo a criar:** `scripts/baixar_geojson_municipios_ne.py` (one-time)

**Como testar:**

```
uv run python -m src.jobs.education_jobs.municipio_pipeline.main
# Deve agregar ~1794 municípios com polígonos
```

**O que estudar:** - API de malhas do IBGE: <https://servicodados.ibge.gov.br/api/docs/malhas> - Formato GeoJSON vs Shapefile — quando usar cada um

---

## 9. Sprint 3 — Indicadores Educacionais Multi-UF + CEP

**Objetivo:** Pipeline educacional completo funcionando para 9 estados (geocodificação + indicadores + agregações).

**Duração:** 2 semanas

**Pré-requisitos:** Sprint 2 concluída (shapefiles disponíveis)

---

### Task 3.1: Expandir filtro do `geocode_pipeline` para NE

**Arquivo a alterar:** `config/config_geocode.yml`

**O que fazer:** 1. Mudar filtro\_uf: ["PB"] → ["MA", "PI", "CE", "RN", "PB", "PE", "AL", "SE", "BA"] 2. Renomear outputs: escolas\_pb.parquet → escolas\_nordeste.parquet, escolas\_pb\_geocoded.parquet → escolas\_nordeste\_geocoded.parquet

**Arquivo a alterar:** src/jobs/education\_jobs/geocode\_pipeline/etl/extract.py - O código já usa `df["SG_UF"].isin(filter_config["filtro_uf"])` — **funciona sem mudança** desde que o config seja lista.

**Impacto:** De 3.737 → ~75.000 escolas processadas no geocode.

**Geocodificação:** O arquivo data/silver/escolas\_nordeste\_geocoded.parquet já tem 49k escolas geocodificadas. As ~26k restantes precisarão de coordenadas via: 1. **Base dos Dados** (latitude/longitude já disponíveis no dataset de escolas) — gratuito 2. **Placeholder (-999, -999)** para escolas sem coordenada — não participam de spatial joins mas ficam no MongoDB

**Como testar:**

```
uv run python -m src.jobs.education_jobs.geocode_pipeline.main
# Deve processar ~75k escolas, ~49k com coords existentes
```

**O que estudar:** - O `_load_existing_geocoded()` no transform — como ele faz merge por CO\_ENTIDADE

---

### Task 3.2: Expandir INEP indicadores para NE

**Arquivo a alterar:** config/inep\_indicadores.yml

**O que fazer:** 1. Mudar filtro\_uf: "PB" → filtro\_uf: ["MA", "PI", "CE", "RN", "PB", "PE", "AL", "SE", "BA"]

**Arquivo a alterar:** src/jobs/education\_jobs/inep\_indicadores\_pipeline/etl/extract.py

**O que fazer:** 1. Na função `_ler_xlsx_inep()`: mudar `df[df["SG_UF"] == filtro_uf]` → `df[df["SG_UF"].isin(filtro_uf)]`  
2. O filtro\_uf vira lista — ajustar onde é usado como string

**Arquivo a alterar:** src/jobs/education\_jobs/inep\_indicadores\_pipeline/etl/transform.py

**O que fazer:** 1. Na `_validar_saida()`: mudar `threshold 3_000` → `30_000` 2. Na validação de UF: `df[df["SG_UF"] != filtro_uf]` → `df[~df["SG_UF"].isin(filtro_uf)]`

**Como testar:**

```
uv run python -m src.jobs.education_jobs.inep_indicadores_pipeline.main
# Deve retornar ~35k escolas com indicadores
```

**O que estudar:** - O config `inep_indicadores.yml` — como `fontes` define as URLs dos XLSXs (são nacionais, não precisam mudar) - Por que o `filtro_dependencias` exclui privadas (foco em escola pública)

---

### Task 3.3: Adquirir e integrar dataset de CEPs do Nordeste

**Arquivo a alterar:** data/gold/cep.json (substituir ou expandir)

**O que fazer:** 1. Adquirir base de CEPs que cubra os 9 estados do NE. Fontes: - OpenCEP (gratuito): <https://github.com/opencep/opencep> - Base dos CEPs (comercial): <https://www.basedosceps.com.br/> - ViaCEP dump (gratuito, requer scraping) 2. Formato esperado pelo `cep_lookup.py`:

```
[
  {"cep": "58000000", "bairro": "Centro", "municipio": "João Pessoa", "id_mundv": "2507507", "sg_uf": "PB", "latitud
```

3. Validar cobertura: quantos CEPs únicos por estado?

**Alternativa se CEP não cobrir tudo:** O `municipio_pipeline` pode usar `CO_MUNICIPIO` do censo (join direto por código IBGE em vez de CEP). Isso eliminaria dependência de CEP para a agregação por município.

**Como testar:**

```
uv run python -c "  
import json  
with open('data/gold/cep.json') as f:  
    ceps = json.load(f)  
import pandas as pd  
df = pd.DataFrame(ceps)  
print(df.groupby('sg_uf').size())  
# Deve ter registros para MA, PI, CE, RN, PB, PE, AL, SE, BA  
"
```

**O que estudar:** - Estrutura de CEPs no Brasil (5 dígitos de prefixo por faixa regional) - O `enriquecer_com_cep()` em `cep_lookup.py` — como faz o left join

---

### Task 3.4: Rodar pipeline educacional end-to-end para NE

Nenhum arquivo novo — é execução e validação.

**O que fazer:** 1. Rodar o pipeline completo:

```
uv run python -m src.jobs.education_jobs.main
```

2. Verificar no MongoDB:

- `escolas`: ~75k documentos com `matriculas.totalAlunos`
- `municipio_indicadores.educacao`: ~1794 municípios com dados
- `bairro_indicadores.educacao`: ~3000 bairros
- `setor_indicadores.educacao`: ~50k setores com escolas

3. Validar integridade:

```
uv run python -c "  
from pymongo import MongoClient  
import os  
client = MongoClient(os.getenv('MONGO_URI'))  
db = client[os.getenv('MONGO_DB_NAME')]  
print('escolas:', db.escolas.count_documents({}))  
print('municipios:', db.municipio_indicadores.count_documents({}))  
print('bairros:', db.bairro_indicadores.count_documents({'educacao': {'\\$exists': True}}))  
print('setores:', db.setor_indicadores.count_documents({'educacao': {'\\$exists': True}}))  
"
```

---

## 10. Sprint 4 — Validação, Testes, Otimização e Deploy

**Objetivo:** Pipeline validado, testado, otimizado e em produção.

**Duração:** 2 semanas

**Pré-requisitos:** Sprint 3 concluída

---

### Task 4.1: Criar testes para os novos indicadores e multi-UF

**Arquivo a criar:** `tests/test_multi_uf_extract.py`

**O que fazer:** 1. Testar que `_filtrar_uf()` com lista de UFs retorna registros de todos os estados 2. Testar que filenames gerados usam `_nordeste.parquet` 3. Mock do FTP para não precisar de rede

**Arquivo a criar:** `tests/test_geo_ingest_multi_uf.py`

**O que fazer:** 1. Testar que o GeoPackage consolidado tem polígonos de todas as 9 UFs 2. Testar que CRS é EPSG:4326 em todos

**Como testar:** `uv run python -m pytest tests/ -v`

**O que estudar:** - `pytest fixtures` e `mocking` (`unittest.mock.patch`) - Como mockar FTP e HTTP calls (`responses library` para requests)

---

#### Task 4.2: Otimizar spatial joins para performance

**Arquivo a alterar:** `src/common/geo_utils.py` e/ou `transforms` de `bairro/setor`

**O que fazer:** 1. Medir tempo e memória do spatial join com NE completo 2. Se RAM > 6GB ou tempo > 5min: implementar chunking por UF 3. Criar índice espacial R-tree explícito: `gdf.sindex` (GeoPandas já faz automático mas podemos forçar)

**Arquivo opcional a criar:** `src/common/spatial_utils.py`

```
def spatial_join_por_uf(gdf_pontos, gdf_poligonos, coluna_uf="SG_UF"):
    """Spatial join chunked por UF para controle de memória."""
    ...
```

**Como testar:**

```
time uv run python -m src.jobs.education_jobs.setor_pipeline.main
# Deve completar em < 5 minutos com < 6GB RAM
```

**O que estudar:** - R-tree spatial index: <https://geopandas.org/en/stable/docs/reference/sindex.html> - `memory_profiler` para medir uso de RAM - `time command` para benchmark de tempo

---

#### Task 4.3: Atualizar Makefile e Docker para NE

**Arquivo a alterar:** `Makefile`

**O que fazer:** 1. Os targets existentes continuam funcionando (nomes genéricos) 2. Adicionar target `run-geo-ingest-all` que baixa shapefiles de 9 UFs 3. Ajustar comentários/help que mencionam PB

**Arquivo a alterar:** `docker-compose.prod.yml`

**O que fazer:** 1. Aumentar memory limit do ETL: 4G → 8G (spatial joins com NE completo) 2. Aumentar volume de dados esperado

**Como testar:** Deploy em staging com `docker compose -f docker-compose.prod.yml run --rm etl python -m src.jobs.education_jobs.main`

---

#### Task 4.4: Script de validação multi-estado

**Arquivo a alterar:** `scripts/validar_socioeconomico.py`

**O que fazer:** 1. Carregar Gold com sufixo `_nordeste.parquet` 2. Validar contagem por UF (loop com benchmarks por estado):

```
BENCHMARKS = {
    "BA": {"municipios": 417, "pop_min": 14_000_000},
    "PE": {"municipios": 185, "pop_min": 9_000_000},
```

```

    "CE": {"municipios": 184, "pop_min": 9_000_000},
    # ... todos os 9
}

```

3. Manter validação de João Pessoa como sanity check + adicionar Salvador e Recife

**Como testar:** `uv run python scripts/validar_socioeconomico.py`

---

### Task 4.5: Deploy em produção

**O que fazer:** 1. Merge da branch para main 2. Build da imagem Docker: `docker compose -f docker-compose.prod.yml build` 3. Rodar pipeline completo no servidor:

```

docker compose -f docker-compose.prod.yml run --rm etl python -m src.jobs.education_jobs.main
docker compose -f docker-compose.prod.yml run --rm etl python -m src.jobs.socioeconomico_jobs.main

```

- Validar no MongoDB Atlas/Express que os dados estão corretos
- Notificar equipe de backend sobre novos dados disponíveis

**Rollback:** Se algo falhar, os dados de PB continuam intactos (upsert não deleta). Basta reverter o deploy e re-executar com config antigo.

---

## 11. Diagrama de Dependências entre Tasks

| Sprint 1        | Sprint 2               | Sprint 3            | Sprint 4           |
|-----------------|------------------------|---------------------|--------------------|
| [1.1 filtro_uf] | -> [2.1 geo_ingest]    | -> [3.1 geocode NE] | -> [4.1 testes]    |
| [1.2 filenames] | -> [2.2 bairro gpkg]   | -> [3.2 INEP NE]    | -> [4.2 perf]      |
| [1.3 loads]     | -> [2.3 setor gpkg]    | -> [3.3 CEP NE]     | -> [4.3 docker]    |
| [1.4 config]    | -> [2.4 municipio geo] | -> [3.4 e2e test]   | -> [4.4 validacao] |
| [1.5 validacao] |                        |                     | [4.5 deploy]       |

---

## 12. Custos Estimados

| Item                        | Custo                 | Observação                                                                |
|-----------------------------|-----------------------|---------------------------------------------------------------------------|
| Geocodificação (Google API) | \$0 – \$125           | Apenas para ~25k escolas sem coord. Fallback gratuito via Base dos Dados. |
| CEP dataset                 | \$0                   | OpenCEP é gratuito                                                        |
| Infra (MongoDB Atlas)       | +\$0                  | 260k docs cabem no tier atual                                             |
| Infra (servidor)            | +\$0                  | Mesmo servidor, apenas mais RAM para ETL                                  |
| Tempo de engenharia         | 4 sprints × 2 semanas | Estimativa conservadora com 1 dev                                         |

---

## 13. Critérios de Aceitação

A expansão é considerada completa quando:

- MongoDB contem dados educacionais para ~75k escolas de 9 estados
- `municipio_indicadores` tem 1.794 documentos com `educacao + socioeconomico`
- `setor_indicadores` tem dados socioeconomicos para ~180k setores
- `bairro_indicadores` tem dados para ~3.000 bairros
- Pipeline executa end-to-end em < 2h (incluindo downloads de shapefiles)

6. Testes automatizados cobrem multi-UF
  7. Nenhum campo `null` onde PB tinha dados (regressao zero)
  8. Backend consome dados sem alteracao de schema
- 

## 14. Referências

- IBGE FTP: [ftp://ftp.ibge.gov.br/Censos/Censo\\_Demografico\\_2022/](ftp://ftp.ibge.gov.br/Censos/Censo_Demografico_2022/)
- IBGE GeoFTP: [https://geoftp.ibge.gov.br/organizacao\\_do\\_territorio/malhas\\_territoriais/](https://geoftp.ibge.gov.br/organizacao_do_territorio/malhas_territoriais/)
- INEP Microdados: <https://www.gov.br/inep/pt-br/aceso-a-informacao/dados-abertos/microdados>
- INEP Indicadores: <https://www.gov.br/inep/pt-br/aceso-a-informacao/dados-abertos/indicadores-educacionais>
- OpenCEP: <https://github.com/opencep/opencep>
- GeoPandas Spatial Joins: [https://geopandas.org/en/stable/gallery/spatial\\_joins.html](https://geopandas.org/en/stable/gallery/spatial_joins.html)
- MongoDB Indexes: <https://www.mongodb.com/docs/manual/indexes/>